Class 4 Notes

The reusable system design concepts behind Search Typeahead — not the solution, but the techniques you'll use in every design

Yesterday's problem was 'design Google-style search autocomplete.' These notes deliberately do NOT walk through that solution again. Instead, they extract the dozen general-purpose ideas we applied — the ones you should recognize and reuse in the next ten problems you face.

1. Decompose the Product Before Designing the System

Never design 'autocomplete.' Decompose it into sub-products, each with its own workload and requirements:

- **Suggest-as-you-type (read path):** prefix in, top-10 out — fired on every keystroke, brutal latency budget.
- **Popularity tracking (write path):** every selection/enter increments a counter somewhere.
- **Ranking (compute):** turn raw counts into 'most popular + trending' — recency matters.
- **Result serving:** on selection, show the links associated with the query.

Each piece can now be designed, scaled, and even owned by different teams independently. Notice the feedback loop: the read path generates the events that feed the write path, which reshapes what the read path serves. Many real systems have this loop (recommendations, trending, ads), and spotting it early tells you where the hard problems live.

This is exactly the pattern from Class 2: the weather platform decomposed into collect / distribute / query. Decomposition is the very first move, every time.

2. How to Break Down Any Problem Statement

A repeatable recipe we used in class:

1. **Identify actors and verbs:** who touches the system (users typing, users selecting) and what they do.
2. **Separate the read path from the write path:** they almost always have different scale, latency, and consistency needs — design them separately, then connect them.
3. **Characterize the workload shape:** read:write ratio, request amplification, burstiness. Typeahead reads are amplified ~20× (one search = ~20 keystrokes = ~20 prefix queries → ~2M read QPS), and one write fans out to N prefix updates. Amplification hides in innocent-looking features.
4. **Pin the latency budget to the interaction:** suggestions must appear between keystrokes — roughly 100 ms end-to-end — which immediately rules out entire categories of design (no disk scans, no cross-region hops on the hot path).
5. **Ask what staleness the product can tolerate:** the answer ('popularity can lag — eventual consistency') unlocked every optimization we made later.

3. Scope Ruthlessly: Goals AND Non-Goals

We wrote down, up front, what we would NOT build: spell correction, personalization, wildcards, query blocking, >50 chars, mixed case. This is not laziness — it's engineering discipline, and it's why real design docs have a non-goals section (Class 1 template).

- Every excluded feature removes a dimension of complexity; declaring exclusions makes the 45-minute problem solvable and shows the interviewer you know the full landscape.
- Constraints simplify the data structure itself: 'lowercase a–z + space only' is what makes a 27-child trie node possible at all.
- In interviews, say it out loud: “I'll treat spell-tolerance as out of scope; if we have time we can revisit.” You get credit for awareness without paying the design cost.

4. Asking High-Quality Clarifying Questions

The test of a good clarifying question: its answer changes your design. If any answer leaves your design unchanged, the question was trivia. The typeahead questions that mattered:

Question	Why it changes the design
Prefix-only, or match anywhere / typo-tolerant?	Prefix-only → trie/prefix keys work. Fuzzy → completely different machinery (n-grams, edit distance, search engine).
How fresh must popularity be? Is 'eventually' OK?	Unlocks (or forbids) sampling, buffering, batch updates — half the design.
Exact counts, or just correct ranking?	Ranking-only → approximation and sampling become legal.
How many suggestions, max query length?	Top-10 and 50 chars bound storage per node and let us precompute answers.
Case sensitivity? Character set? Languages?	Defines the alphabet — the branching factor of the whole data structure.
What happens on selection?	Reveals the write path and the feedback loop — easy to miss entirely.
Scale: QPS, users, growth?	Decides single-machine vs sharded, RAM vs disk.

> **A pattern for generating good questions**
>
> Walk one user interaction end to end in your head, and at every step ask: what's the input space? (charset, length) — what's the output contract? (how many, ranked how) — how fresh? (consistency) — how fast? (latency) — how often? (scale). Five axes × each step ≈ all the questions that matter.

5. Estimate Before You Architect — and Use Estimates to Compare Designs

We didn't estimate for ritual; the numbers made decisions:

- Storage: flat strings ≈ 11 TB/year vs trie ≈ 13.7 TB/year (with 75% prefix overlap). Lesson: a 'space-saving' structure isn't automatically smaller — pointers cost bytes too. We accepted slightly more storage to buy much faster prefix reads.
- QPS: ~100K write QPS and — because of keystroke amplification — ~2M read QPS. Those two numbers alone dictate 'read path must be memory-speed and horizontally scaled' and 'write path must be tamed.'
- General habit: when torn between two designs, estimate both. Numbers end debates that adjectives can't.

6. Sometimes You Must Build Your Own Data Structure

General-purpose databases index data for general-purpose queries. When your access pattern is narrow and extreme — 'top-10 completions for a prefix, in under 100 ms, at 2M QPS' — a purpose-built structure can beat any generic index. Hence the trie: the path from root to a node IS the prefix; lookup cost depends on prefix length, not on how many billions of strings are stored.

Then we customized it further — and this is the deeper lesson. The classic trie answers 'what strings have this prefix?' but our product question is 'what are the 10 BEST strings for this prefix?' So we changed the structure to answer the product question directly: store the precomputed top-10 list on every node. Reads became O(prefix length + 10); writes got heavier. That's not a textbook trie — it's OUR trie, shaped by OUR requirement.

You are allowed — expected — to do this. Famous purpose-built structures that exist for exactly this reason:

Structure	Narrow question it answers	Used in
Trie (+ top-K per node)	Best completions for a prefix	Autocomplete, IP routing tables
Inverted index	Which documents contain this word?	Elasticsearch, every search engine
Bloom filter	Is this key definitely absent?	Cassandra/HBase (skip disk reads), CDNs
LSM tree	Absorb massive sequential writes	Cassandra, RocksDB, LevelDB
HyperLogLog / count-min sketch	Approximate distinct-count / frequency in tiny memory	Redis, analytics, trending
Geohash / quadtree / R-tree	What's near this point?	PostGIS (Class 2!), Uber, maps
Consistent-hash ring	Which node owns this key, with minimal reshuffling?	Dynamo, Cassandra, caches

7. Precompute: Move Work from Read Time to Write Time

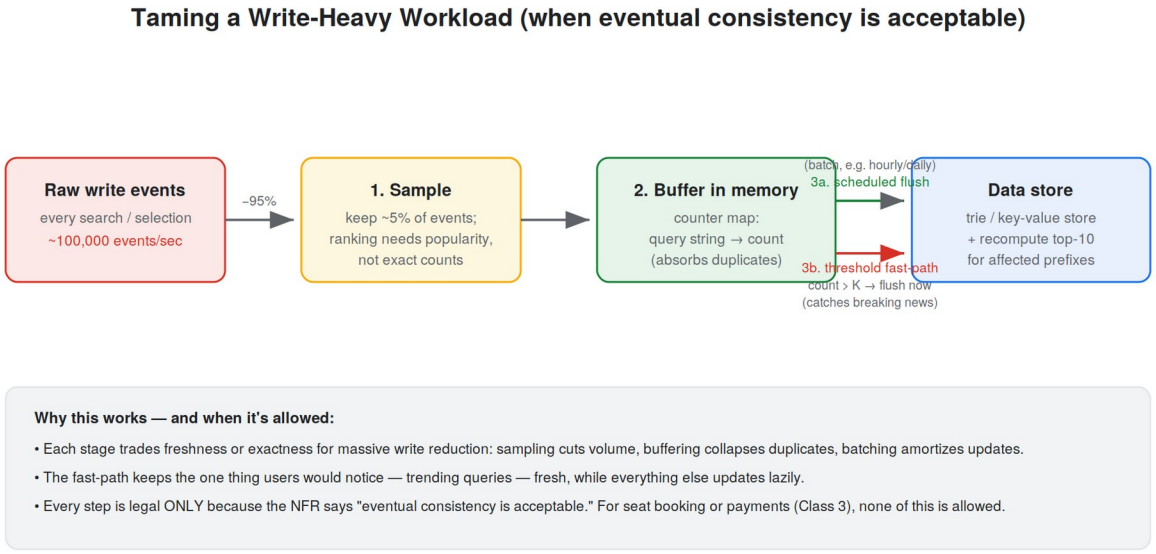
Storing top-10 on every node is one instance of the most important performance idea in system design: if reads vastly outnumber writes (2M vs 100K here — and remember the 100:1 ratios of Classes 2–3), do the expensive work once at write time and make every read a cheap lookup.

- Typeahead: recompute top-10 lists when counts change → reads never search or sort.
- Weather platform (Class 2): precomputed daily aggregates → the API never scans raw readings.
- Social feeds: fan-out-on-write pushes a new post into followers' precomputed timelines → opening the app is one read.
- Databases call it a materialized view; caches are the degenerate case (precompute = remember the last answer).

The eternal trade-off: write amplification and storage in exchange for read speed. You earn the right to pay that price by knowing your read:write ratio — which is why we estimate first.

8. Buffering + Sampling + Thresholds: Taming the Write Path

With one write fanning out to N prefix updates, the naive write path collapses — and reads suffer too, because they contend with writes on the same structure. Because the NFR said 'eventual consistency is acceptable,' we got to use a three-stage funnel:



- **Buffer (batch):** accumulate counts in an in-memory map; flush to the real store on a schedule. Thousands of increments to 'cricket score' collapse into one +N update. Real-world versions: metrics pipelines (StatsD), YouTube view counters, like counts, log aggregation — and Kafka itself is essentially this buffer as a service, decoupling event producers from slow consumers (Class 2's ingestion queue).
- **Sample:** count only ~5% of events. Legal because the product needs ranking, not bookkeeping — relative popularity survives sampling. Real-world versions: distributed tracing (trace 1% of requests), A/B metrics, network flow monitoring.

- **Threshold fast-path:** pure batching has a product bug — breaking news wouldn't trend until tomorrow's flush. Fix: if a buffered count crosses a threshold, flush it immediately. General pattern: batch by default, express lane for outliers. You'll meet it again in payment risk (score async, block obvious fraud inline) and alerting (aggregate logs, page instantly on FATAL).

> **The discipline**
>
> Each of these three moves silently drops a guarantee (freshness, exactness, immediacy). That's only allowed because a requirement explicitly permitted it. Always be able to point at the NFR that pays for the optimization — in a seat-booking or payment system (Class 3's PC side), all three are forbidden.

9. Time Decay: When 'Popular' Must Mean 'Popular NOW'

Raw lifetime counters have a bug: a query that got a billion searches during one event stays ranked #1 forever. The fix is exponential decay — periodically divide every score by a decay factor, so old popularity fades unless refreshed by new activity:

Day	New searches for “corona virus”	Score (new + previous ÷ 1.2)
Jan 1	10,000	10,000
Jan 2	1,000	1,000 + 10,000/1.2 = 9,333
Jan 3	100	100 + 9,333/1.2 = 7,877
Jan 4	10	10 + 7,877/1.2 = 6,574

Tune the factor to tune memory: a large divisor forgets fast (great for 'trending now'), a small one preserves long-term classics. Real-world appearances of the same idea:

- Hacker News / Reddit ranking: score ÷ (age)^gravity — stories sink as they age no matter how many upvotes they collected.
- Twitter/X trending topics: velocity of mentions, heavily decayed — yesterday's hashtag is nothing.
- Fraud & credit risk scores: old incidents matter exponentially less than last week's.
- Cache eviction (LFU with aging): decay frequency counts so yesterday's hot key can actually be evicted.
- Monitoring: exponentially weighted moving averages (load average in Linux is exactly this).

The unifying idea: recency is information. Whenever a score feeds a 'right now' decision, build forgetting into the math.

10. Sharding Lessons: Hot Shards, Cold Shards, and Who Keeps the Map

Once the structure outgrows one machine, how you split it matters more than that you split it:

- Naive range/prefix sharding (a–z → 26 machines; 3-char prefixes → 26^3 shards) produces skew: 'how' is scorching hot, 'zzz' is frozen. Real data is never uniform — expect a power law.
- Mitigations: pack many cold shards onto one machine, give hot shards big machines (or split them further), and keep the prefix→shard mapping in a coordination service like ZooKeeper — which you can now classify instantly: PC/EC (Class 3), because a wrong map is worse than a briefly unavailable one.
- Consistent hashing spreads prefixes pseudo-randomly around a ring, evening out load and minimizing data movement when nodes join/leave — the trade: you lose range locality.
- Interview takeaway: choose the partition key by access distribution, not by what's alphabetically neat. Then say how you'd detect and handle the inevitable hot shard.

11. Tiering: Put the Hot 20% in RAM, the Cold 80% on Disk

80–90% of queries are short — so keep the top few levels of the trie (short prefixes) in memory and let deep, rarely-touched levels live on disk. Memory answers most traffic; disk cost stays sane. This hot/cold tiering is everywhere: cache in front of database, S3 Standard → Infrequent Access → Glacier lifecycle rules (your Class 1 cloud homework), OS page caches, CPU cache hierarchies. The recipe: measure the access distribution, find the knee, split the storage tiers there.

12. Reshape Your Data So a Boring, Proven Database Can Serve It

The distributed trie works — but WE own the sharding, rebalancing, hot-spot management, replication, and failure recovery forever. That is enormous, permanent engineering overhead for a feature, not a product.

The alternative we explored: restate the problem as key→value. Key = the prefix itself ('sys', 'syst', 'syste', …); value = the precomputed top-10 list. Suddenly any battle-tested horizontally-scalable KV store (DynamoDB, Cassandra, Redis Cluster) handles sharding, replication, and scaling for us — problems those teams have spent a decade solving.

The price, stated honestly: storage blows up (every prefix of every string is materialized) and one write touches N keys (write amplification again). We pay storage and write cost — the two things we already know how to tame (Sections 7–8) — to eliminate an entire category of operational risk.

> **The principle (memorize this one)**
>
> Custom infrastructure must justify itself against 'reshape the data to fit a managed store.' The best design is usually not the theoretically optimal one — it's the one your team can operate at 3 a.m. Ask: can I turn my access pattern into get(key)/put(key)? Into a time-ordered append? Into a geo query PostGIS already does (Class 2)? If yes, the 'boring' choice usually wins.

Related trick with the same spirit and its own costs: dictionary encoding — map strings to integers and store integer lists to shrink memory and speed comparisons. Real systems (columnar stores, analytics engines) do this everywhere; at Google scale it strains ID space and demands collision handling, so we noted it and moved on. Not every optimization survives contact with the scale estimate — which is fine; saying why you rejected something is design work too.

13. How to Communicate During a System Design Interview

Notice HOW the class session itself was structured — that structure is the communication technique:

6. **Propose → attack → repair, out loud.** We gave the naive trie, found the sort-at-read flaw, fixed it with top-10 precompute, found the write flaw, fixed it with buffering… Interviewers score this iteration loop far higher than a memorized final answer, because it shows you can think, not recite.
7. **Signpost your route.** "Requirements → estimates → API → high-level → deep dive" — announce it, then announce transitions. The interviewer relaxes when they can see the map.
8. **Attach every decision to a requirement or a number.** Not "I'll use a cache" but "reads are 2M QPS and 100 ms budget, so the hot path must be memory-resident." The word 'because' is your highest-scoring word.
9. **State assumptions and invite correction.** "I'm assuming ~20% of queries are new each day — fair?" Wrong-but-stated beats silently-wrong every time.
10. **Name the trade-off at the moment you make it.** "Batching improves throughput but delays trending — here's the threshold fast-path to compensate." Pros AND cons, unprompted.
11. **Use precise shared vocabulary.** Read/write amplification, eventual consistency, PA/EL, hot shard, precompute, decay — the vocabulary from these classes compresses ideas and signals fluency.
12. **Park rabbit holes explicitly.** "Collision handling here is a solvable detail — I'll flag it and stay on the critical path unless you want to go deeper." You control the clock politely.

14. Summary: Concept → Typeahead → Where You'll See It Next

Concept	In typeahead	Elsewhere
Product decomposition	Suggest / track / rank / serve	Every design (weather: collect/distribute/query)
Read-write path separation	2M read QPS vs 100K write QPS	CQRS, replicas, feeds
Request amplification	20× reads (keystrokes), N× writes (prefixes)	Chat fan-out, cache stampedes
Custom data structures	Trie with top-10 per node	Bloom filters, LSM trees, inverted indexes
Precompute at write time	Top-10 stored, never sorted at read	Materialized views, feed fan-out
Write buffering / batching	In-memory count map, periodic flush	Metrics, view counters, Kafka
Sampling / approximation	Count 5% of traffic	Tracing, HyperLogLog, analytics
Threshold fast-path	Trending flushes immediately	Fraud checks, alerting
Time decay	Score ÷ 1.2 daily	HN/Reddit ranking, risk scores, EWMA
Skew-aware sharding	Hot 'how', cold 'zzz'; consistent	Any partitioned store; celebrity problem

Concept	In typeahead	Elsewhere
	hashing	
Hot/cold tiering	Top trie levels in RAM	Caches, S3 storage classes
Fit a boring DB	Prefix→top-10 in a KV store	The default question for ALL custom infra

The meta-lesson

A 'simple autocomplete box' forced out a dozen deep techniques — and we had to CONSTRAIN the problem hard just to keep it tractable (real Google adds personalization, spelling, every language, and availability so good people use google.com to test their Wi-Fi). Master the concepts, not the solutions: next class's problem will be different, but it will be built from these same twelve moves.